Software Engineering

Chapter 1: Introduction to Software Engineering

Instructor: Professor Nien-Lin Hsueh (with Gemini AI)
Department of Information Engineering and Computer Science
Feng Chia University

Chapter 1: Roadmap & Core Curriculum

Part 1: Historical Genesis & Foundations

- **1.1 Software Changes the World:** From room-sized mainframes to ubiquitous AI and digital infrastructure.
- **1.2 The Genesis of SE:** The 1968 NATO Conference, software crisis, and catastrophic historical failures.
- **1.3 What is Software?:** Beyond raw code: programs, data structures, procedures, and documentation.
- **1.4 What is Engineering?:** Real-world trade-offs: balancing competing constraints under finite resources.
- **1.5 What is Software Engineering?:** Formal definition, 4 core activities, SWEBOK, and modern toolchains.

Part 2: Quality, Ethics & Modern AI Era

- **1.6 Software Quality Models:** ISO/IEC 25010 product quality characteristics and operational attributes.
- **1.7 Professional Ethics & Dark Patterns:** ACM/IEEE Code of Ethics, public interest, and deceptive UI design.
- **1.8 AI in Software Engineering:** Paradigm shifts, SDLC lifecycle impact matrix, and technical engineering rigor.
- **1.9 Synthesis & Conceptual Recap:** Key takeaway review, FAQ, interactive quiz, and classic references.

1.1 Software Changes the World

> *"Software is eating the world."*
> *— Marc Andreessen*

Software Changes the World: From Mainframes to PCs (1950s–1980s)

- **The Mainframe & Minicomputer Era (1950s–1970s):**
  - *The Paradigm:* Giant machines occupying climate-controlled rooms (IBM System/360); software fed via punch cards and magnetic tapes.
  - *Human Impact:* Automated government censuses, defense radar, and core banking ledgers—computations that previously took months of human calculation were completed in hours.
- **The Personal Computer (PC) Revolution (1980s):**
  - *The Paradigm:* Microprocessors brought software onto every desktop (DOS, Apple Macintosh, Windows, VisiCalc, Lotus 1-2-3).
  - *Daily Convenience:*
    - Transformed paper-based offices into interactive digital workspaces.
    - Real-time electronic spreadsheets and word processing freed billions of worker hours from tedious manual recalculation and retyping.

The Connected World: The Web & Internet Revolution (1990s–2000s)

- **The Birth of the World Wide Web (1990s):**
  - *The Paradigm:* Tim Berners-Lee created HTTP/HTML; browsers (Netscape, Internet Explorer) transformed the Internet into a universal, clickable cyberspace.
  - *Human Impact:* **Democratization of Global Information**—encyclopedias, academic research, and global news became instantly searchable via Yahoo and Google.
- **The Rise of E-Commerce & Global Platforms (2000s):**
  - *The Paradigm:* Secure web transactions (SSL), distributed databases, and web services (Amazon, eBay, PayPal, Wikipedia).
  - *Daily Convenience:*
    - **Elimination of Geographic Distance:** Instant global communication via email and instant messaging.
    - **24/7 Digital Commerce:** Shopping, ticket booking, and online banking replaced physical lines and postal delays.

The Pocket Revolution: Mobile & Cloud Everywhere (2010s)

- **Smartphones & The App Economy (iOS & Android):**
  - *The Paradigm:* Software moved into our pockets, tightly integrated with GPS, cameras, accelerometers, and high-speed cellular networks (4G/5G).
  - *Human Impact:* **Software as an extension of the human body**—over 6 billion people carry supercomputers everywhere they go.
- **Cloud Computing & The On-Demand Economy:**
  - *The Paradigm:* Elastic cloud infrastructure (AWS, Azure) streaming software, media, and computation on demand.
  - *Daily Convenience:*
    - **Living in the Cloud:** Ride-hailing (Uber), food delivery, on-demand streaming (Spotify, Netflix), and instant cashless mobile payments.
    - **Frictionless Navigation & Living:** Real-time traffic rerouting (Google Maps) and seamless remote collaboration tools.

The Cognitive Leap: Generative AI & Autonomous Systems (2020s–Present)

- **From Deterministic Logic to Probabilistic Intelligence:**
  - *The Paradigm:* Large Language Models (LLMs), multi-modal foundation models, and autonomous AI agents (ChatGPT, GitHub Copilot, Gemini).
  - *Human Impact:* Software no longer just executes pre-written rules—it understands natural language, generates code, synthesizes media, and reasons across complex knowledge domains.
- **Ambient Intelligence & Cyber-Physical Autonomy:**
  - *The Paradigm:* AI integrated with physical vehicles (self-driving cars), robotics, smart grids, and medical precision diagnostics.
  - *Daily Convenience:*
    - **Zero Marginal Cost of Cognitive Assistance:** Instant 24/7 personal tutors, coding copilots, and multi-lingual real-time translation.
    - **Accelerated Scientific Discovery:** AI predicts protein structures (AlphaFold) and accelerates new drug design from decades to weeks.

1.2 The Genesis of Software Engineering

> *"The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! As long as there were no machines, programming was no problem at all."*
> *— Edsger W. Dijkstra (1972 Turing Award Lecture)*

The Software Crisis of the Late 1960s

- As hardware costs plummeted, software demand and complexity exploded.
- **The Symptoms of Crisis:**
  - Chronic budget overruns (often 3x–4x initial estimates).
  - Critical project delays and abandoned deliveries.
  - Severe defects, system crashes, and unmaintainability.
- **The Core Problem:** Informal, craft-like programming does not scale to large teams and complex systems.

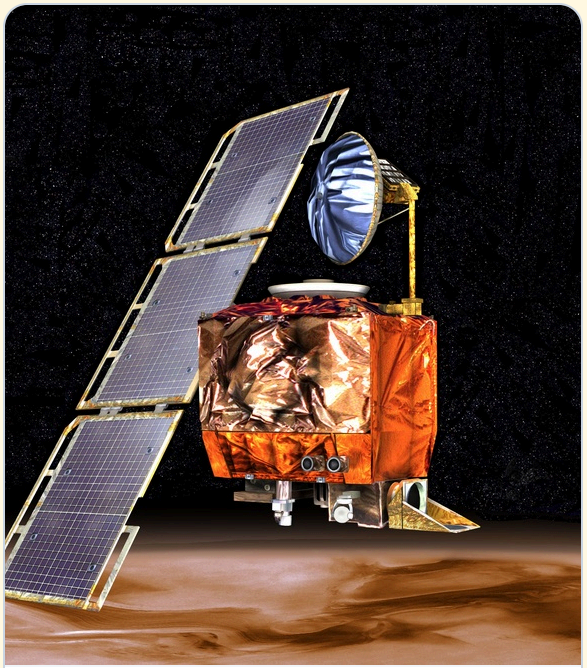


The 1968 NATO Garmisch Conference

- **October 1968 in Garmisch, Germany:**
  - 50 leading computer scientists and industry managers convened.
  - Formally established the term **"Software Engineering"**.
- **The Mission:**
  - Transition software development from an undisciplined, artisanal craft into a **formal engineering discipline**.
  - Establish structured methodologies, rigorous cost estimation, formal verification, and project management.

The High Cost of Software Failure

- **Nagoya Airbus A300 Crash (1994):**
  - Autopilot Go-Around mode stayed active; fought pilot's manual steering, leading to trim nose-up stall and 264 fatalities.
- **Mars Climate Orbiter (1999):**
  - $327M probe lost because ground software calculated thrust in imperial $lbf \cdot s$, while onboard computer expected metric $N \cdot s$.
- **Ariane 5 Flight 501 (1996):**
  - 64-bit float horizontal velocity overflowed 16-bit signed integer ($> 32,767$), crashing processors in 37 seconds.



Mars Climate Orbiter

NASA / JPL / Wikipedia

Concept Check: The Software Crisis (CCQ 1)

Why couldn't the 1968 Software Crisis be resolved simply by purchasing faster computer hardware or larger memory?

- **A.** Computer hardware manufacturing and memory fabrication completely stagnated in the late 1960s, preventing computational speedups.
- **B.** The crisis was fundamentally an intellectual and organizational challenge of system complexity, which faster hardware only amplified.
- **C.** Programming languages of that era strictly lacked mathematical calculation primitives and compiler memory allocation capabilities.
- **D.** Early mainframe computers were physically incompatible with shared telecommunication networks and multi-terminal architectures.



We need Engineering method to develop Software

- What is **Software** ?
- What is **Engineering** ?
- What is **Software Engineering** ?

1.3 What is Software?

> *"Source code is only the tip of the software iceberg. Programs, data, documentation, and configuration together breathe life into a system."*

The IEEE Anatomy of Software

> ***Software (IEEE Standard):*** *Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.*

- **1. Programs:** Executable binaries and source code files (Python, Java, C++, TypeScript).
- **2. Data & Schemas:** Database tables, config files, AI weights (e.g. Knight Capital's config error).
- **3. Operational Procedures:** Deployment scripts, backups, recovery runbooks (e.g. GitLab's backup failure).
- **4. Documentation:** Architecture ADDs, API specs (OpenAPI), user guides (e.g. Therac-25 undocumented bugs).

The Software Iceberg Trap: Beyond Raw Source Code

- **The Amateur Fallacy:**
  - Novices and shortsighted managers treat software as merely the visible tip—**Programs (Source Code)**—measuring progress solely by lines of code written.
- **The Hidden Reality (80%+ Below the Surface):**
  - In production systems, over **80% of effort, complexity, and catastrophic failures** reside below the surface in Data, Procedures, and Documentation:
    - *Knight Capital ($440M lost in 45 min):* Core code was fine, but a flawed deployment **procedure** and config **data** caused bankruptcy.
    - *GitLab Database Outage:* Production binaries worked, but disaster recovery **procedures** were untested.
    - *Therac-25:* Inadequate architectural **documentation** concealed lethal concurrency bugs.
- *Without all four pillars, a system is not "engineered software"—it is merely a brittle program.*

Why the Four Pillars Demand the Software Life Cycle (SDLC)

> ***The Core Epiphany: Programming*** *is writing code in an afternoon.* ***Software Engineering*** *is governing all four pillars across the entire* ***Software Life Cycle (SDLC)****.*
>
> ”

- **1. Documentation $\longleft→ Requirements & Architectural Design:**
  - Code only records *how*, but documentation (ADRs, OpenAPI, specs) captures *why*. Without architectural contracts, multi-year team collaboration and system evolution are impossible.
- **2. Programs $\longleft→ Implementation & Automated Verification:**
  - Writing code is only one brief phase. Software engineering mandates continuous unit testing, CI/CD verification, and refactoring to prevent code rot and technical debt.
- **3. Data & Schemas $\longleft→ State Persistence & Long-Term Evolution:**
  - Code changes in seconds, but data lives for decades. The lifecycle requires strict schema versioning, backward compatibility, and database migration engineering.
- **4. Procedures $\longleft→ DevOps, Deployment & Site Reliability (SRE):**
  - Code on `localhost` is a toy. Engineering requires automated deployment pipelines, canary rollouts, disaster recovery runbooks, and telemetry monitoring.

Why the 4 Pillars Matter: The YouBike Example

- **1. Programs (Execution Engines):**
  - *Embedded lock firmware, mobile apps, and cloud backend microservices.*
  - Execute real-time lock/unlock protocols, GPS tracking, and concurrent user requests reliably.
- **2. Data & Schemas (Single Source of Truth):**
  - *Bike locations, dock availability, user account balances, and ACID rental logs.*
  - Data corruption or unsynced records cause "ghost bikes", erroneous billing, and lost revenue.
- **3. Operational Procedures (System Resilience):**
  - *firmware updates, dock rebalancing logistics, payment failover, and battery triage.*
  - A flawed OTA deployment bricks thousands of street locks, requiring costly physical field servicing.
- **4. Documentation (Collaboration Contracts):**
  - *OpenAPI specs, MQTT telemetry protocols, and hardware-software interface contracts.*
  - Prevents integration breakdowns among hardware vendors, mobile teams, and municipal transit.

Concept Check: The Software Definition (CCQ 2)

According to the IEEE standard definition of software, which of the following is NOT considered a component of software?

- **A.** Executable computer programs and source code files.
- **B.** System database schemas and configuration files.
- **C.** CPU processor hardware and physical memory units.
- **D.** Software installation and deployment procedures.



1.4 What is Engineering?

> *"Scientists discover the world that exists; engineers create the world that never was."*
> *— Theodore von Kármán*

WHAT IS ENGINEERING?

FOUNDATIONS & RESOURCES



Scientific Principles

$$f(x) = \frac{m - \mu}{\pi\, m} \qquad f_{\text{n}} = = \frac{m}{\pi}$$

$$f_0 = \mu^{12} + e^{n/2} \qquad E = mb^2$$

Empirical Methods & Experience

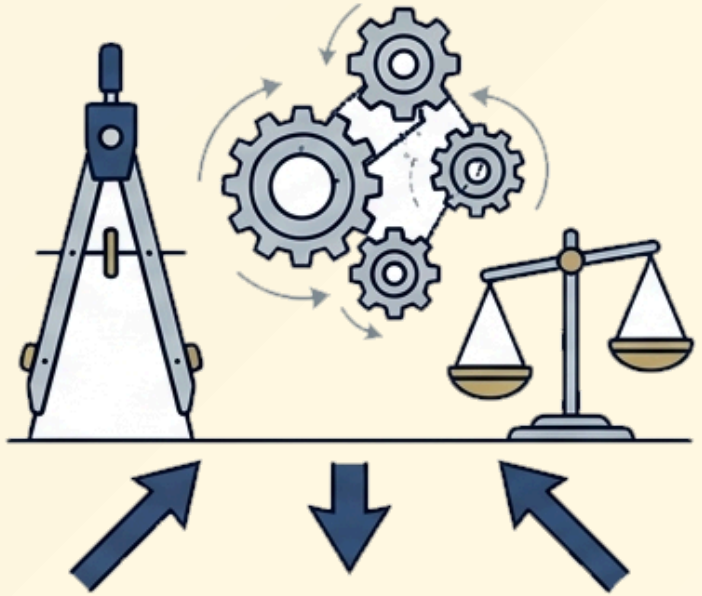


Finite Resources



Finite Resources

CREATIVE DESIGN & CONSTRAINED OPTIMIZATION



Real-World Constraints

Cost Limits
Deadlines
Safety Regulations

USEFUL ARTIFACTS & SOLUTIONS



Inventing & Constructing for Societal Needs

What is Engineering?

> ***Engineering:*** *The creative application of scientific principles, empirical methods, and practical experience to invent, design, and construct useful artifacts under* ***real-world constraints*** *with* ***finite resources****.*

- **Three Core Hallmarks of Professional Engineering:**
  - **1. Systematic:** Following structured, repeatable processes and methodologies rather than ad-hoc trial and error.
  - **2. Disciplined:** Adhering strictly to industry standards, rigorous peer reviews, and safety margins.
  - **3. Quantifiable & Measurable:** Evaluating success and stability through empirical metrics (latency, reliability, test coverage, uptime).
- **Engineering Under Constraints:**
  - Engineering is never done in an ideal vacuum—it is the art of **constrained optimization**, balancing goals against finite time, budget, and resources.

The Engineering Equation: Constraints vs. Resources

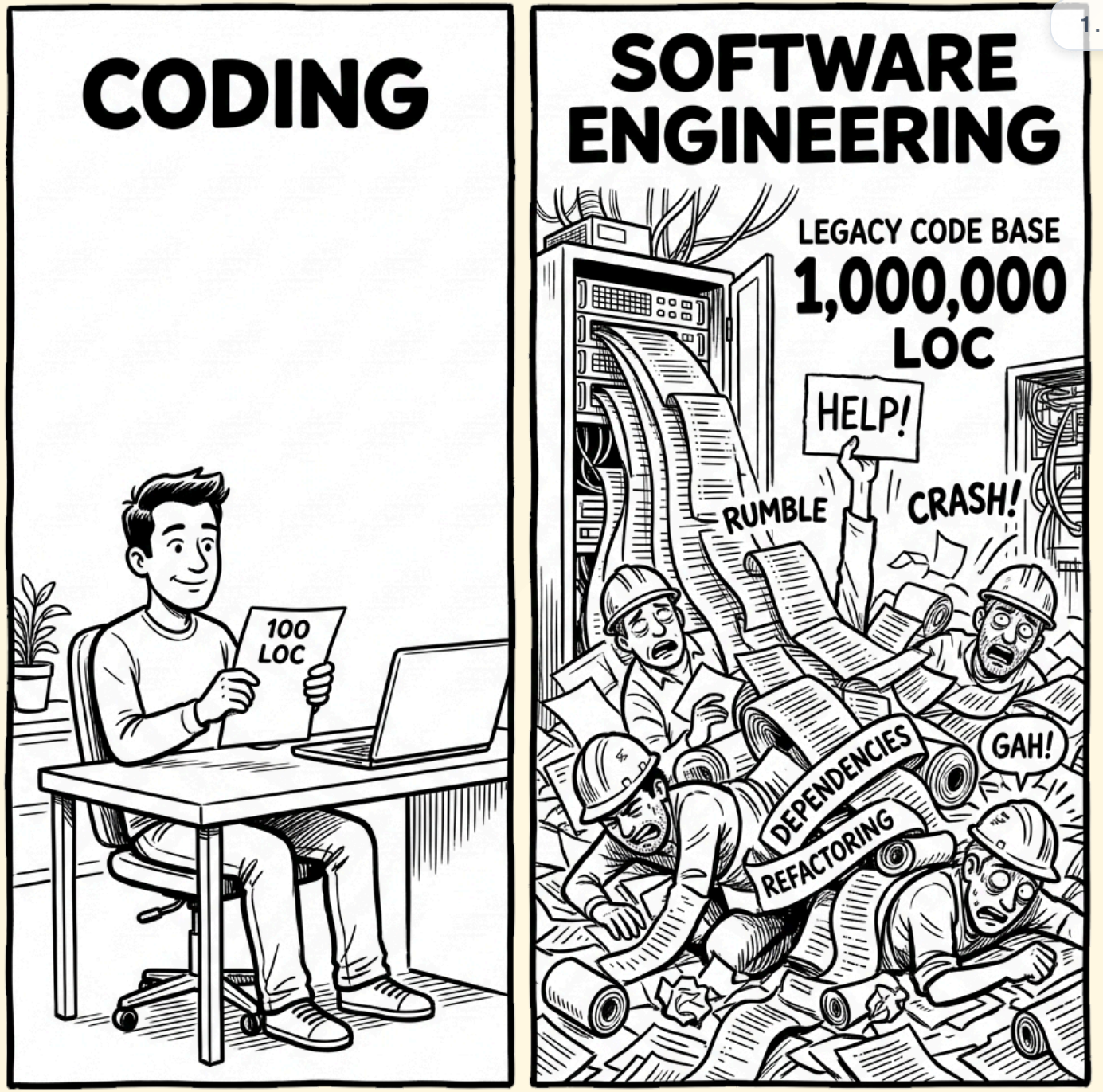
- **Constraints (The Limitations):**
  - **Time:** Hard deadlines, release windows.
  - **Budget:** Developer salaries, cloud hosting bills, license fees.
  - **Technology & Platform:** Legacy databases, mobile OS limits.
  - **Regulations:** GDPR data privacy, HIPAA, PCI-DSS.
- **Resources (The Assets):**
  - **Human:** Developer skill, QA engineers, UX designers.
  - **Tools & Infrastructure:** Cloud compute, CI/CD pipelines, open-source libraries.
  - **Domain Knowledge:** Understanding user workflows and business logic.

Case Study: Healthcare Startup MVP (Minimun Viable Product)

- **Problem:** Launch a HIPAA-compliant telemedicine app in 6 months on a tight $80k budget.
- **The Engineering Balancing Solution:**
  1. **Scope Prioritization:** Focus MVP strictly on video consults and booking; postpone insurance billing.
  2. **Cross-Platform Tooling:** Use Flutter/React Native for a single codebase (saves 40% effort).
  3. **Managed Backend:** Use HIPAA-compliant cloud BaaS instead of bare-metal servers.
  4. **Automated CI/CD:** unit tests on PRs, keeping QA overhead low.
- **Why Not "Perfect" Technical Solutions?**
  - Custom native apps and custom microservices are technically superior, but rejected because they violate constraints ($80k budget, 6 months).
  - **Satisficing:** Find the solution that meets all constraints through pragmatic trade-offs.
  - **The Engineering Imperative:** We need **Requirements Engineering** to negotiate scope against finite budgets, and **System Design** to negotiate trade-offs that satisfy real-world constraints.

1.5 What is Software Engineering?

> *"Software engineering is programming integrated over time, with teams, under changing constraints."*
> *— Titus Winters (Software Engineering at Google)*

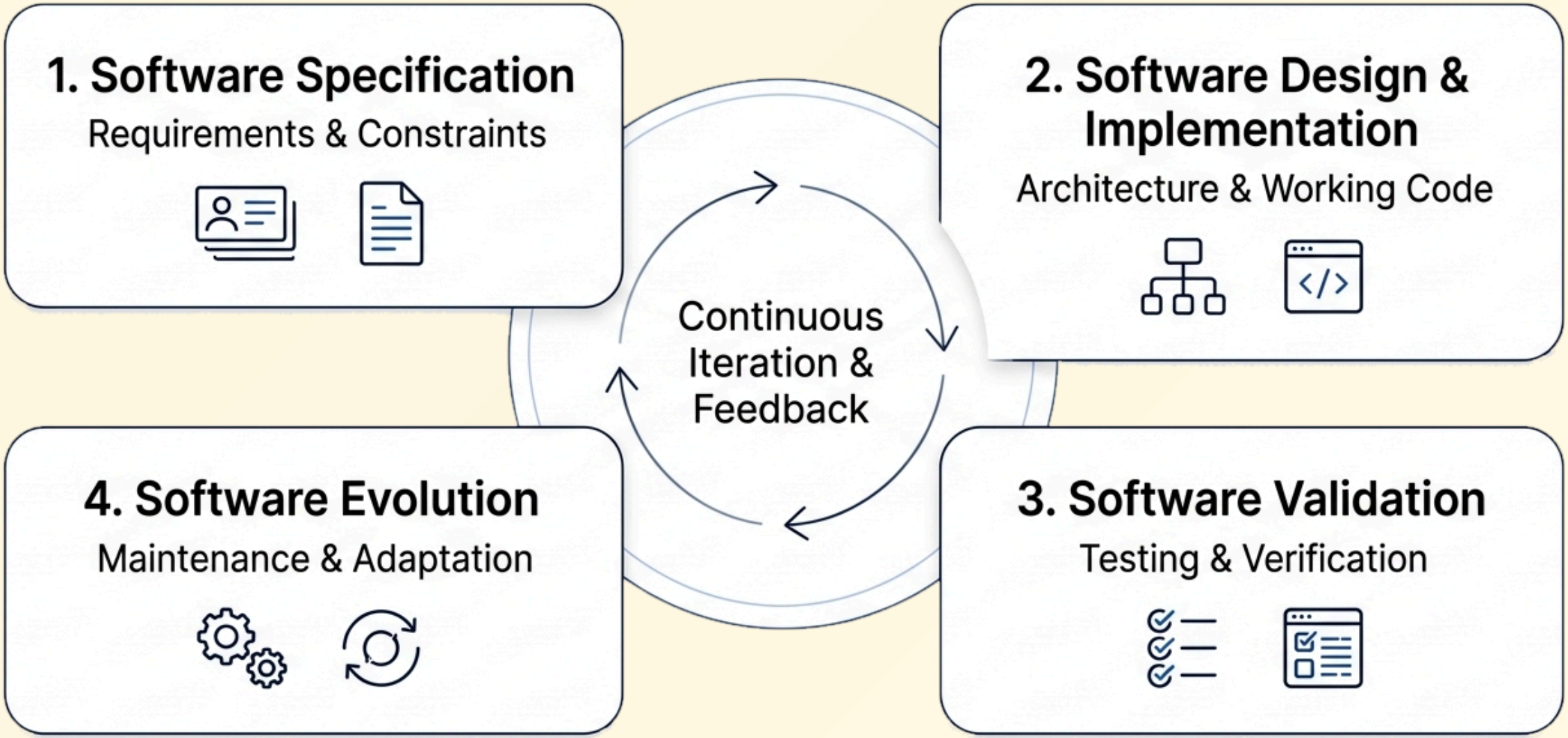

CODING
100 LOC
SOFTWARE ENGINEERING
LEGACY CODE BASE
1,000,000 LOC
HELP!
RUMBLE
CRASH!
DEPENDENCIES
REFACTORING
GAH!

What is Software Engineering?

> ***Software Engineering (IEEE 610.12):*** *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.*

- **1. Engineering Discipline (Systematic, Disciplined, Quantifiable):**
  - Applies scientific rigor, engineering standards, and empirical metrics rather than ad-hoc craftsmanship.
- **2. Full Lifecycle Scope (Development, Operation, Maintenance):**
  - **Development:** Initial specification, architectural design, coding, and quality verification.
  - **Operation & Maintenance:** Ensuring 24/7 runtime stability, performance optimization, and long-term system evolution.

The Software Engineering Process: Four Universal Activities



Activity 1: Software Specification

> ***Defining what the system must do and the constraints under which it must operate.***
>
> ”

- **Core Mission:**
  - Discover, negotiate, and establish user needs and system boundaries before building.
- **Key Lifecycle Sub-Activities:**
  - **Feasibility Study:** Is the project technically and economically viable?
  - **Requirements Elicitation & Analysis:** Discovering what stakeholders actually need.
  - **Specification & Validation:** Documenting requirements into testable, unambiguous contracts.
- **Key Artifacts:** User Stories, Acceptance Criteria (Given-When-Then), SRS, Use Cases.
- **The High-Stakes Risk:**
  - Building the wrong product! A requirement defect caught in production costs **up to 100x more** to resolve than if caught during specification.

Activity 2: Software Design & Implementation

> ***Translating requirements into executable software structures and source code.***
>
> ”

- **Core Mission:**
  - Transform abstract specifications into robust, maintainable, and high-performance software.
- **Key Lifecycle Sub-Activities:**
  - **Architectural Design:** Defining overall system structure, services, and subsystems.
  - **Interface & Data Modeling:** Establishing modular API contracts and database schemas.
  - **Component Design & Coding:** Writing clean, testable logic, algorithms, and modules.
- **Key Artifacts:** Architecture Decision Records (ADRs), ERDs, OpenAPI Specs, Clean Code.
- **The High-Stakes Risk:**
  - Spaghetti architecture, tight coupling, and runaway technical debt that make future modifications prohibitively expensive.

Activity 3: Software Validation

> ***Checking that the software conforms to its specification and satisfies customer needs.***
>
> ”

- **The Twin Pillars of Quality (Boehm):**
  - **Verification:** "*Are we building the product right?*" (Conforming strictly to specification).
  - **Validation:** "*Are we building the right product?*" (Meeting real customer intent).
- **Key Lifecycle Sub-Activities:**
  - **Unit & Component Testing:** Isolating modules and verifying boundary behaviors.
  - **Integration & System Testing:** Verifying inter-service contracts and end-to-end flows.
  - **Acceptance Testing & Code Review:** Peer review audits and user sign-off.
- **Key Artifacts:** Automated Test Suites (PyTest, JUnit), CI Pipeline Reports, Coverage Metrics.
- **The High-Stakes Risk:**
  - Critical production outages, catastrophic security breaches, data loss, and legal liability.

Activity 4: Software Evolution

> ***Modifying and adapting deployed software to meet changing business and user needs.***
>
> ”

- **The Reality of Software Economics:**
  - Real-world software is never "done." Over **60%–80% of total lifecycle cost** occurs in Evolution after initial launch!
- **The Four Modes of Maintenance:**
  - **Corrective:** Fixing latent defects, crash bugs, and edge cases in production.
  - **Adaptive:** Migrating to new cloud runtimes, mobile OS updates, or changing regulations.
  - **Perfective:** Improving throughput, latency, UX responsiveness, and adding new features.
  - **Preventive:** Continuous refactoring to eradicate technical debt before failures occur.
- **Key Artifacts:** Database Migration Scripts, Release Notes, Incident Post-Mortems.

Concept Check: Core Activities (CCQ 3)

Which of the following pairs correctly matches a specific software engineering action with its corresponding universal core activity?

- **A.** Conducting stakeholder interviews to draft user stories → Software Specification
- **B.** Writing automated unit tests to mock database responses → Software Design & Implementation
- **C.** Refactoring database schemas to improve query speed → Software Validation
- **D.** Swapping a third-party payment API for a new gateway → Software Specification



CORE ELEMENTS OF SOFTWARE ENGINEERING

Disciplines (Best Practices)

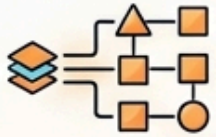
Spec before design
Design before code



- Interface-based
- Change management
- ADR (Architecture Decision Records)
- Code Reviews

Principles (Core Foundations)

- Abstraction
- Modularity



- Anticipation of Change
- Captration



Open-Closed (OCP)
KISS (Keep It Simple, Stupid)
POLA (Principle of Least Astonishment)

Methods & Methodologies



- Waterfall
- Agile / Scrum
- Spiral Model
- TDD (Test-Driven Development)
- CI/CD & DevOps

Heuristics & Guidelines



- Nielsen's 10 UX Heuristics
- SOLID design
- Clean Code
- DRY (Don't Repeat Yourself)
- YAGNI (You Ain't Gonna Need It)

The Software Engineering Body of Knowledge (BOK)

- Software Engineering is defined by a structured **Body of Knowledge (BOK)**—a system of proven **engineering heuristics** and principles designed to conquer complexity:
  - **Disciplines (Rules of Practice):** Mandatory professional habits and operational protocols ("*Spec before design*", "*Design before code*", *Architecture Decision Records*, *Code reviews*).
  - **Principles (Foundations):** Timeless, enduring engineering truths (*Abstraction, Modularity, Separation of Concerns, Anticipation of Change*).
  - **Methods & Methodologies:** Structured, repeatable process frameworks (*Agile/Scrum, Extreme Programming, TDD, CI/CD pipelines, DevOps*).
  - **Heuristics & Guidelines:** Practical rules of thumb and design heuristics distilled from decades of field experience (*SOLID, Clean Code, KISS, DRY, YAGNI, POLA*).

Applying the Body of Knowledge to the 4 Core Activities

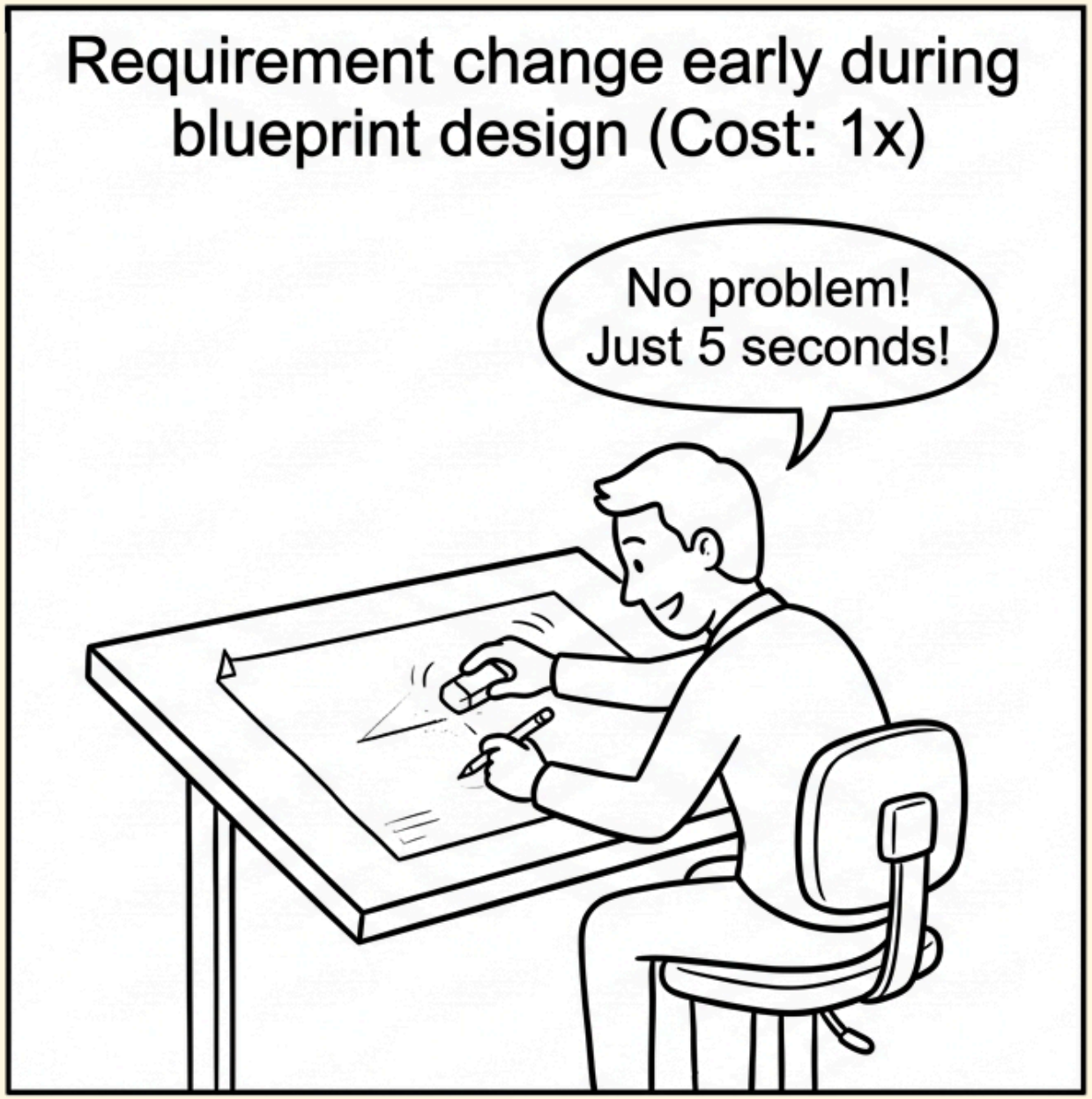
- The Body of Knowledge is not abstract theory—it directly anchors each of the **Four Universal Lifecycle Activities**:
- **1. Specification (Requirements):**
  - *Disciplines & Principles:* **"Spec before design"**, Abstraction, Separation of Concerns.
  - *Heuristics:* **YAGNI** (prevent speculative features), unambiguous acceptance criteria.
- **2. Design & Implementation:**
  - *Disciplines & Principles:* **"Design before code"**, Modularity, Information Hiding, ADRs.
  - *Heuristics:* **SOLID principles**, Clean Architecture, GoF patterns, **DRY**, **KISS**.
- **3. Validation (Testing):**
  - *Disciplines & Principles:* **TDD**, independent verification, mandatory Peer Code Reviews.
  - *Heuristics:* Boundary value testing, defense-in-depth, automated regression safety nets.
- **4. Evolution (Maintenance):**
  - *Disciplines & Principles:* **Anticipation of Change**, Semantic Versioning, Technical Debt tracking.
  - *Heuristics:* **The Boy Scout Rule** (*leave code cleaner than you found it*), zero-downtime rollouts.

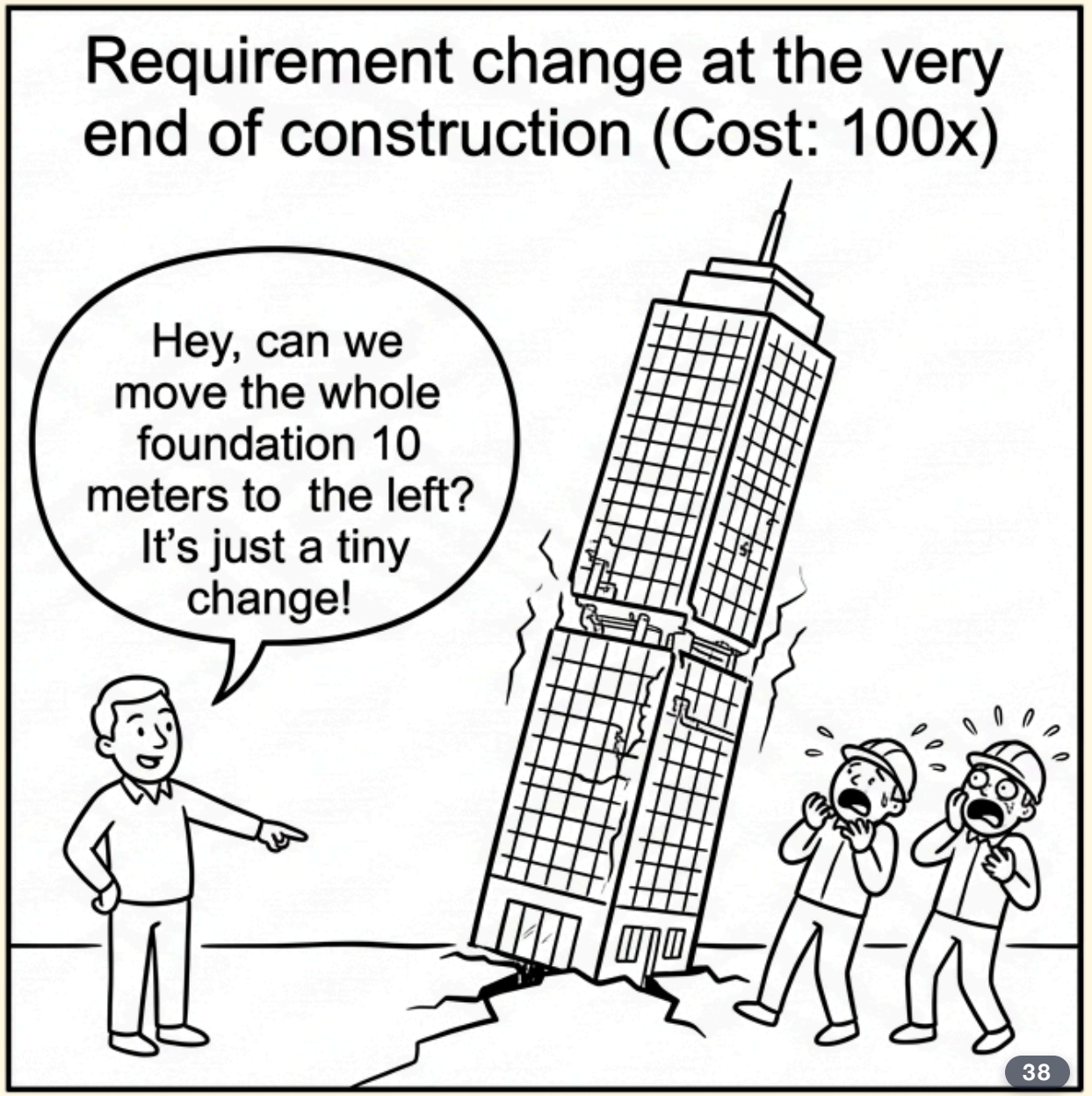
Why Principles Matter

> ***When engineering principles are ignored, intuition leads to expensive fallacies.***

- **Myth 1: "We're behind schedule—let's add 5 developers to catch up."**
  - *Violates:* **Modularity & Communication Boundaries**.
  - *Reality (Brooks's Law):* Adding manpower to a late project makes it later due to $O(n^2)$ communication overhead.
- **Myth 2: "Software is digital, so changing requirements late is cheap."**
  - *Violates:* **"Spec Before Design" & Architectural Coupling**.
  - *Reality:* Late changes invalidate schemas, API contracts, and tests, costing **up to 100x more**.
- **Myth 3: "Outsource the coding and we don't need technical management."**
  - *Violates:* **The Software Iceberg (Data, Procedures, Docs)**.
  - *Reality:* Raw code without architectural governance creates unmaintainable software debt.

Myth: Changing requirements late is cheap!





Concept Check: Brooks's Law (CCQ 4)

A project is 3 weeks behind schedule with 2 weeks remaining before release. The manager hires 4 junior programmers to speed up progress. What will happen according to Brooks's Law?

- **A.** The project will finish 1 week early.
- **B.** The project will be delayed further because senior engineers must spend time onboarding and mentoring new hires.
- **C.** The existing developers will code twice as fast.
- **D.** Communication complexity remains unchanged.



Concept Check: Design Principles (CCQ 5)

An order-processing module directly handles HTTP requests, executes payment transactions, queries the SQL database, and generates HTML receipt emails. Which fundamental design principle is most severely violated?

- **A.** Separation of Concerns: Multiple distinct responsibilities are tightly tangled in a single module.
- **B.** YAGNI: Speculative future features are implemented before actual business requirements emerge.
- **C.** Brooks's Law: Adding developers to the order module increases communication complexity exponentially.
- **D.** Anticipation of Change: System configurations are hardcoded into compiled production binaries.



Enforcing Disciplines at Scale: The Modern Toolchain

> ***Engineers cannot enforce disciplines by willpower alone—tools automate the BOK.***
>
> ”

- **Version Control (Git) → *Enforces Collaboration & Review*:** Branching models, PR audits, and history.
- **CI/CD Pipelines (GitHub Actions) → *Enforces Continuous Verification*:** Automated build, test, and lint gates.
- **Static Code Analysis (SonarQube) → *Enforces Clean Code & Security*:** Detecting code smells and CVEs.
- **Testing Suites (PyTest, Playwright) → *Enforces Quality Standards*:** Unit, integration, and E2E regression nets.
- **Observability (OpenTelemetry, APM) → *Enforces Evolution Feedback*:** Real-time production telemetry and traces.

1.6 The ISO/IEC 25010 Quality Standard

> *"Quality is value to some person. If a system runs without error but nobody wants it, it has zero quality."*
> *— Jerry Weinberg*

ISO/IEC 25010 Software Product Quality Model



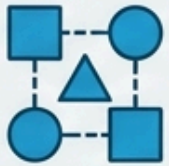
Functional Suitability

- Functional Completeness
- Functional Correctness
- Functional Appropriateness



Performance Efficiency

- Time Behaviour
- Resource Utilization
- Capacity



Compatibility

- Co-existence
- Interoperability



Usability

- Appropriateness Recognizability
- Learnability
- Operability
- User Error Protection
- User Interface Aesthetics
- Accessibility



Reliability

- Maturity
- Availability
- Fault Tolerance
- Recoverability



Security

- Confidentiality
- Integrity
- Non-repudiation
- Accountability
- Authenticity



Maintainability

- Modularity
- Reusability
- Analysability
- Modifiability
- Testability



Portability

- Adaptability
- Installability
- Replaceability

What is Quality and Quality Model?

> ***Software Quality:*** *The degree to which a software product satisfies stated and implied needs of its stakeholders under specified conditions.*

- **What is Software Quality? (Beyond "No Bugs"):**
  - Novices mistakenly equate quality with "*my code compiles and doesn't crash on my laptop.*"
  - True quality is multi-dimensional: software can run without runtime crashes while still being unmaintainable, insecure, horribly slow, or unusable.
  - Encompasses both **External Quality** (user experience, speed, uptime) and **Internal Quality** (code maintainability, modularity, test coverage).
- **What is a Software Quality Model?:**
  - **Structured Multi-Dimensional Taxonomy:** Decomposes abstract "quality" into clear **characteristics**, **sub-characteristics**, and **measurable metrics**.
  - **Concrete Engineering Contract:** Translates ambiguous business wishes ("*make the app fast and secure*") into testable quantitative targets ($p99 < 100\text{ms}$, *zero high-severity CVEs*).
  - **Guide for Architecture Trade-Offs:** Clarifies which quality attributes take priority (*Safety over Time-to-Market*, or *Portability over Raw Performance*).

The Modern Standard: ISO/IEC 25010 (SQuaRE)

> ***ISO/IEC 25010 (Software product Quality Requirements and Evaluation): The internationally recognized benchmark standard for specifying and evaluating software quality.***

- **Key Characteristics & Modern Architecture:**
  - **Comprehensive 8-Characteristic Taxonomy:** Systematically covers Functional Suitability, Performance, Compatibility, Usability, Reliability, Security, Maintainability, and Portability.
  - **First-Class Security:** Elevates Security to an independent, top-tier pillar (Confidentiality, Integrity, Non-repudiation, Accountability, Authenticity).
  - **Emphasis on Compatibility & Interoperability:** Treats multi-platform co-existence and standardized API interoperability as primary requirements in distributed/cloud systems.
- **Dual-Perspective Quality Architecture:**
  - **Product Quality Model:** 8 intrinsic technical characteristics of the system (for developers, architects, and QA engineers).
  - **Quality in Use Model:** Real-world human impact during operation (*Effectiveness, Efficiency, Satisfaction, Freedom from Risk, Context Coverage*).

ISO 25010: Product Quality Characteristics (1/2)

- **1. Functional Suitability:** *Functions meet stated and implied user needs.*
  - *Sub-attributes:* **Completeness**, **Correctness**, **Appropriateness**.
- **2. Performance Efficiency:** *Performance relative to the amount of resources used.*
  - *Sub-attributes:* **Time Behaviour** (latency, throughput), **Resource Utilization**, **Capacity**.
- **3. Compatibility:** *Ability to share environments and exchange info without conflict.*
  - *Sub-attributes:* **Co-existence** (clean co-habitation), **Interoperability** (data/API exchange).
- **4. Usability (Interaction Capability):** *Ease with which users achieve goals with satisfaction.*
  - *Sub-attributes:* **Recognizability**, **Learnability**, **Operability**, **Error Protection**, **Aesthetics**, **Accessibility**.

ISO 25010: Product Quality Characteristics (2/2)

- **5. Reliability:** *Ability to maintain a specified level of performance over time.*
  - *Sub-attributes:* **Maturity** (low defect rate), **Availability**, **Fault Tolerance**, **Recoverability**.
- **6. Security:** *Protecting data and systems from unauthorized access or tampering.*
  - *Sub-attributes:* **Confidentiality**, **Integrity**, **Non-repudiation**, **Accountability**, **Authenticity**.
- **7. Maintainability:** *Effectiveness and ease with which software can be evolved and fixed.*
  - *Sub-attributes:* **Modularity**, **Reusability**, **Analyzability**, **Modifiability**, **Testability**.
- **8. Portability (Flexibility):** *Ease with which software is transferred across environments.*
  - *Sub-attributes:* **Adaptability**, **Installability**, **Replaceability**.

ISO 25010 Sub-Attributes: Practical Real-World Scenarios

- **Fault Tolerance (Reliability):** Primary payment gateway times out → system retries with backup gateway without dropping the customer transaction.
- **Integrity & Authenticity (Security):** JWT authorization tokens cryptographically signed with RS256 to prevent tampering.
- **Time Behavior & Capacity (Performance Efficiency):** E-commerce search API processes $10,000\ \text{req/sec}$ with $p99 < 80\text{ms}$.
- **Interoperability (Compatibility):** Weather service exposes OpenAPI 3.0 and gRPC contracts for zero-friction client integration.
- **Testability & Modularity (Maintainability):** Code structured with Dependency Injection so databases can be easily mocked in unit tests.
- **Installability (Portability):** Complete local microservices stack spins up in 60s via `docker compose up`.

Concept Check: Software Quality Factors (CCQ 6)

Which of the following matches a real-world software issue with its corresponding ISO 25010 quality characteristic?

- **A.** A database query taking 15 seconds to return results → Maintainability (Testability)
- **B.** A system crash occurring when a third-party API goes offline → Reliability (Fault Tolerance)
- **C.** Developers struggling to write unit tests due to tight coupling → Portability (Adaptability)
- **D.** An unencrypted session cookie allowing account takeover → Usability (Operability)



Interactive Activity: Quality Trade-Off Poll

Quality Trade-off Poll & Discussion:

- **System A:** Hospital ICU Automated Insulin Pump Controller
- **System B:** Mobile Casual Viral Game
- **Poll:** What are the top 2 non-negotiable ISO 25010 attributes for System A vs. System B?
- **Key Question:** Why is prioritizing *Time to Market* over *Fault Tolerance* fatal for System A, but acceptable for System B? In your opinion, what constraints or factors impair software quality or make high quality hard to achieve?

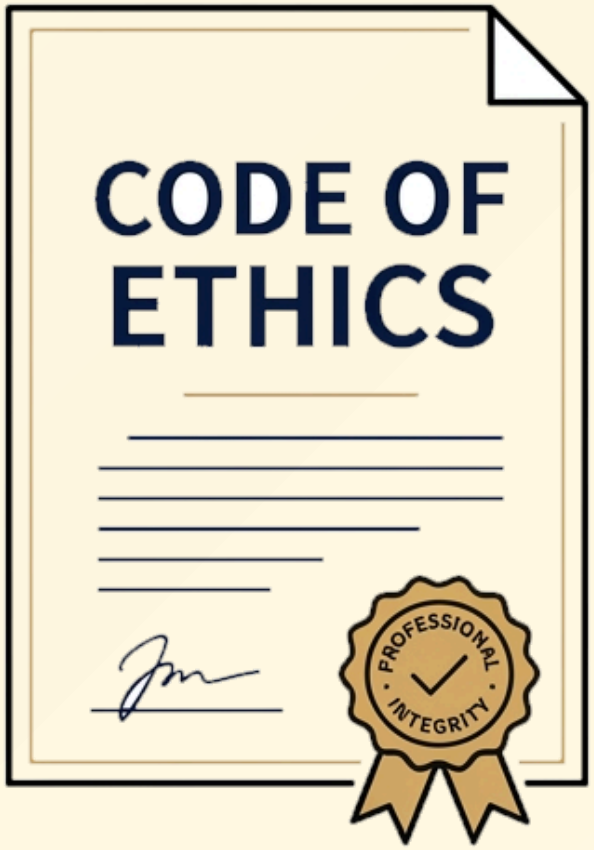


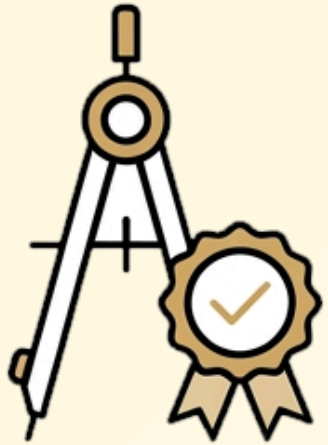
1.7 Professional Ethics and Dark Patterns

> *"With great computational power comes profound ethical responsibility."*



Moral Duties





Professional Standards



Public Accountability

What is a Professional Code of Ethics?

> ***A Code of Ethics is a formal covenant establishing the moral duties, professional standards, and public accountability of a discipline.***

- **The Moral Weight of Software Engineering:**
  - Software is no longer just code—it directly governs human health, aviation safety, elections, and global finance.
  - Unlike casual programmers, **professional engineers hold a fiduciary duty to society**.
- **Why Software Engineers Need an Explicit Code of Ethics:**
  - **Asymmetry of Information:** Users and managers cannot audit millions of lines of code—they must trust the engineer's integrity.
  - **Armor Against Compromise:** When corporate managers push to cut safety tests, fake benchmarks, or ship spyware, the Code provides an authoritative shield.
  - **The Fundamental Anchor: The Public Interest, Safety, and Welfare must always take absolute precedence over employer loyalty.**

ACM/IEEE Software Engineering Code of Ethics: Eight Core Principles

1. PUBLIC



2. CLIENT & EMPLOYER



3. PRODUCT



High-quality software

4. JUDGMENT



5. MANAGEMENT



6. PROFESSION



7. COLLEAGUES



8. SELF



ACM/IEEE Code of Ethics: 8 Core Principles

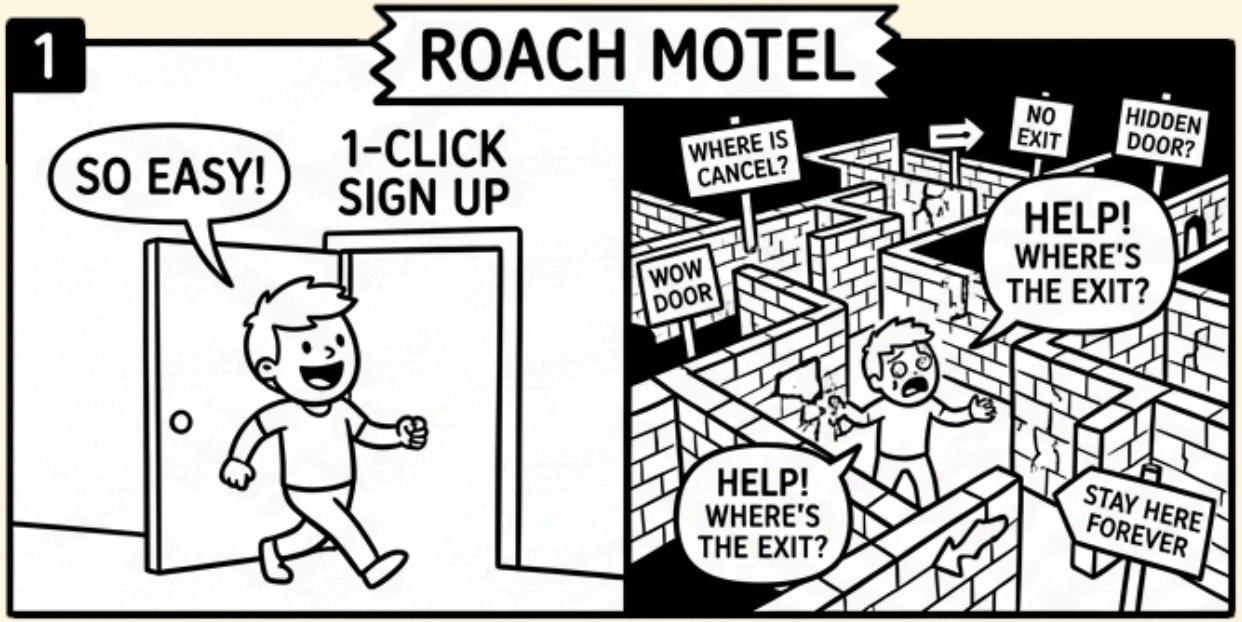
1. **Public:** Prioritize public safety, health, and welfare above all.
2. **Client & Employer:** Act in their best interest, consistent with public interest.
3. **Product:** Ensure software meets high professional standards.
4. **Judgment:** Maintain integrity and independence in technical evaluation.
5. **Management:** Promote ethical management and realistic project estimates.
6. **Profession:** Advance the integrity and reputation of software engineering.
7. **Colleagues:** Be fair to, support, and mentor peers.
8. **Self:** Participate in lifelong learning and ethical practice.

High-Profile Ethical Breaches

- **Volkswagen "Dieselgate" (2015):**
  - Engineers wrote engine software to detect laboratory test cycles and hide toxic NO_x emissions (up to 40x legal limit on the road).
  - Resulted in billions in fines, criminal convictions, and severe environmental harm.
- **Cambridge Analytica (2018):**
  - Improper harvesting of personal data for covert political manipulation.
- **Planned Obsolescence:**
  - Software updates engineered to artificially degrade legacy device performance.

DECEPTIVE UI/UX DARK PATTERNS

1 ROACH MOTEL



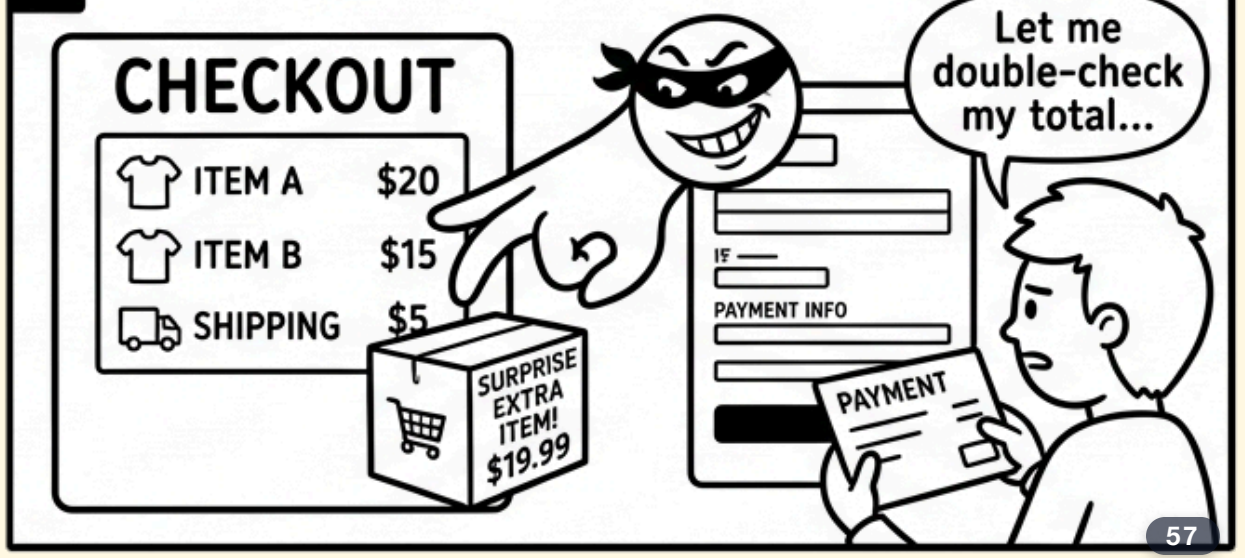
2 CONFIRMSHAMING



3 FAKE URGENCY



4 SNEAK INTO BASKET



Deceptive "Dark Patterns" in UI/UX Design

- **1. Roach Motel (Subscription Labyrinth):**
  - Signing up takes 1 click; cancelling requires navigating hidden menus or making a phone call.
- **2. Confirmshaming:**
  - Emotionally manipulative text on decline buttons ("*No thanks, I hate saving money*").
- **3. Hidden Costs & Sneak into Basket:**
  - Pre-ticking add-on insurance or fees at the final checkout step.
- **4. Fabricated Urgency & Scarcity:**
  - Fake countdown timers ("*Only 2 minutes left!*") and fabricated demand alerts.

Concept Check: Engineering Ethics (CCQ 7)

Under the ACM/IEEE Code of Ethics, if an employer directs an engineer to implement an algorithm that falsifies safety compliance reports, what is the engineer's obligation?

- **A.** Comply, because the employer pays the engineer's salary.
- **B.** Refuse and escalate, because the Public Interest takes precedence over Employer loyalty.
- **C.** Implement the code but omit documentation.
- **D.** Outsource the code to an external vendor.



Interactive Activity: Dark Pattern Detective (Pair Discussion)

Dark Pattern Detective Activity (Pair Discussion with Your Classmate):

- **Identify & Classify:** Recall a deceptive interface you encountered on a real-world app or website. **Which type of dark pattern does it belong to?** (e.g., Roach Motel, Confirmshaming, Sneak into Basket, Fake Urgency, etc.)
- **Ethical Analysis:** Which ACM/IEEE ethical principle (Public Interest, Product Quality, Professional Judgment) was violated?
- **Redesign:** How would you redesign that interaction to achieve legitimate business conversion while remaining transparent, honest, and user-respecting?



1.8 AI in Software Engineering

> *"AI amplifies our velocity, but engineering discipline ensures we are heading in the right direction."*


JUST COMPILE IT.
TRUST THE VIBES!
BREAK GLASS IN CASE OF VIBE CHECK
CRITICAL SYSTEM FAILURE
PRODUCTION SERVERS
LOGS: ERROR 500
LOGS: ERROR 500
CRITICAL OVERLOAD!
KERNEL PANIC!
FIRE IN DATA CENTER!
CODE & VIBES

What is "Vibe Programming"?

> ***"Vibe Coding" (popularized by Andrej Karpathy):***
> *A workflow where developers prompt an AI to generate code, rarely inspecting or understanding the implementation, and "just vibe with whatever runs."*

- **The Seductive Appeal of the "Vibe":**
  - **Zero-Friction Prototyping:** Anyone can prompt an LLM to build a working prototype or full-stack MVP in an afternoon.
  - **The Illusion of Velocity:** Compiling code creates a false sense of production readiness.
- **Why Vibe Coding Fails as Software Engineering:**
  - **Hidden Fragility:** Unchecked AI code harbors subtle concurrency race conditions, security vulnerabilities (CVEs), and edge cases.
  - **Unmaintainable Debt:** When 20,000 lines built on "vibes" break in production, no human understands the causality.



Andrej Karpathy
Wikimedia Commons / Wikipedia

The Hidden Hazards: Three Empirical Problems of AI Coding

- **1. Maintainability Degradation & High Code Churn (GitClear 150M LOC Study):**
  - **Code Duplication Surges:** Copy-paste logic increases; proactive refactoring ("*moved lines*") drops sharply.
  - **High Code Churn:** Code is rapidly deleted or rewritten within two weeks, accumulating massive **Maintainability Debt**.
- **2. 52% Error Rate & The "Illusion of Security" (Purdue University Study):**
  - In 517 software engineering questions, **52% of ChatGPT code answers contained errors**.
  - Due to AI's articulate, confident, and polite tone, **39.3% of developers still preferred and accepted** the flawed code without verification.
- **3. 40% Known Security Vulnerabilities (NYU / Stanford Research):**
  - Automated CWE scans show that without explicit security constraints, $\approx 40\%$ **of AI-generated code** harbors CWE Top 25 vulnerabilities (SQL injection, buffer overflow, race conditions).

Real-World Incidents of Vibe Coding

- **1. Production Outages (Amazon Checkout):** AI-generated patch deployed without complete verification broke checkout logic, dropping orders by 99% and losing **6.3M orders in hours**.
- **2. Slopsquatting (Package Hallucination):** LLMs hallucinate fake package names (`huggingface-cli`); hackers register malware, poisoning builds (30k+ downloads).
- **3. Secrets Sprawl & Hardcoded API Keys:** AI generates sample code with hardcoded passwords and tokens; GitGuardian reports AI code leaks credentials at $2\times$ **the rate of humans**.

The Other Side: Benefits & Real-World Triumphs of AI Coding

- **The Genuine Benefits of AI Assistance:**
  - **1. Eliminating Cognitive Drudgery:** Automates boilerplate, regex, CRUD patterns, and scaffolding—allowing engineers to focus on architectural trade-offs and domain logic.
  - **2. Accelerating Developer Velocity & Flow:** Empirical studies show developers complete routine tasks up to **55% faster**, preserving mental energy and deep focus.
- **Two Landmark Real-World Success Stories:**
  - **Case 1: Amazon's 30,000 Java Upgrades (Amazon Q Developer):**
    - Amazon used AI agents to migrate over 30,000 production applications to Java 17 in months, saving **4,500 developer-years** of manual labor and **$260M in annual efficiency gains**.
  - **Case 2: Enterprise Productivity Surge (Accenture & GitHub Copilot):**
    - Across thousands of enterprise engineers, routine feature delivery accelerated by **55%**, with 90% reporting greater flow state when paired with rigorous peer code reviews.
- **The Engineering Takeaway:**
  - **Harness & govern—neither blindly embrace nor dogmatically reject!** The goal is not to surrender to the "vibe", nor to ban AI out of fear, but to **manage AI coding through engineering discipline, verification, and architectural oversight**.

AI in SWE: The Paradigm Shift (Software 1.0 → 3.0)

- **The Evolution of How We Build Software:**
  - **Software 1.0 (Code-Centric):** Humans write explicit, deterministic algorithms line-by-line (f(x) → y).
  - **Software 2.0 (Prompt-Driven):** Humans write natural language prompts and instructions; LLMs generate code, functions, and boilerplate (e.g., Copilot, ChatGPT).
  - **Software 3.0 (Agentic):** Humans specify high-level goals and architectural constraints; autonomous AI agents iteratively plan, execute tools, run tests, and refactor code.
- **The Fundamental Transformation of the Software Engineer:**
  - **What AI Commoditizes:** Boilerplate syntax, standard CRUD endpoints, and syntax translation.
  - **What Remains Irreplaceable:** Domain analysis, architectural trade-offs, security boundaries, and **evaluating whether generated solutions actually meet user needs**.
  - **The Engineer's New Identity:** Moving from *syntax typist* → **System Architect, Specification Designer & Verification Authority**.

AI in SWE: Requirements & System Architecture

- **1. Requirements Engineering (Specification):**
  - **Superpowers (⊕):** Rapidly drafts user stories, Given-When-Then acceptance criteria, and edge-case scenarios; spots ambiguities and contradictions in specs.
  - **Pitfalls & Risks (⊖):** Hallucinates non-existent APIs and false business logic; lacks organizational tacit knowledge, legal liability context, and human empathy.
- **2. Architectural & System Design:**
  - **Superpowers (⊕):** Compares architectural patterns and trade-offs systematically; rapidly scaffolds ERDs, database schemas, and OpenAPI contracts.
  - **Pitfalls & Risks (⊖):** Promotes premature over-engineering and microservice sprawl; blind to operational latency, infrastructure cost limits, and security SLAs.

AI in SWE: Code Construction & Implementation

- **3. Coding & Implementation:**
  - **Superpowers ($\oplus$):**
    - **Eliminates Boilerplate:** Automates repetitive scaffolding, CRUD endpoints, and complex regex queries.
    - **Polyglot Acceleration:** Translates algorithms and logic seamlessly across programming languages.
    - **Instant Rubber-Ducking:** Explains cryptic compiler errors and suggests alternative implementations in seconds.
  - **Pitfalls & Risks ($\ominus$):**
    - **The "Vibe Coding" Trap:** Developers accept syntactically plausible code without understanding runtime causality.
    - **Security Vulnerabilities:** NYU/Stanford studies show $\approx 40\%$ of AI code contains CWE Top 25 vulnerabilities (SQL injection, buffer overflows, concurrency race conditions).
    - **Supply Chain Poisoning:** Introduces hallucinated packages, deprecated APIs, or restrictive copyleft licenses.

AI in SWE: Validation, Testing & Evolution

- **4. Software Validation (Testing & QA):**
  - **Superpowers (⊕):** Synthesizes comprehensive mock datasets, boundary unit tests, and property-based fuzzing suites.
  - **Pitfalls & Risks (⊖): "Echo-Chamber Testing"**—AI writes tests that merely validate its own flawed assumptions ("*Who tests the tester?*"); unit tests pass on `localhost` but fail under real concurrency.
- **5. Software Evolution (Maintenance):**
  - **Superpowers (⊕):** Deciphers and explains 10-year-old legacy spaghetti codebases in seconds; automates changelogs, docstrings, and migration scripts.
  - **Pitfalls & Risks (⊖):** Introduces **silent regression bugs** during refactoring; generates articulate, authoritative documentation that describes what the code *should* do, not what it *actually* does.

Engineering Rigor in the AI Era

> *"Never merge code you do not understand and cannot defend."*

- **The "Automation Bias" Hazard:**
  - Over-trusting fluent, confident AI outputs without verifying boundary behaviors, race conditions, or edge-case security contracts.
- **Three Imperatives for AI-Augmented Engineering:**
  - **1. Specification-First (Contract-Driven):** Never generate code without formal interface definitions, type signatures, and clear acceptance criteria.
  - **2. Independent Verification Net:** AI cannot write both the implementation and its own test cases unchecked ("*Who tests the tester?*"). Mandate deterministic CI regression suites.
  - **3. Human Professional Accountability:** AI provides drafts, but the human engineer owns **100% of the legal, ethical, and architectural liability** for every line in production.

Concept Check: AI Coding & Code Churn (CCQ 8)

In empirical studies evaluating AI coding assistants (such as GitClear's analysis of 150M lines of code), "Code Churn" emerged as a major warning sign. What does high Code Churn indicate in an AI-assisted codebase?

- **A.** Code is rapidly rewritten, deleted, or patched shortly after commit, indicating brittle code accepted without sufficient verification.
- **B.** Compilers and bundlers are aggressively removing unreachable dead code from application binaries during automated deployment.
- **C.** Software engineering teams are switching programming languages frequently due to automated polyglot syntax translation.
- **D.** Automated test cases are executing too quickly and depleting available CI/CD pipeline virtual machine compute resources.



Concept Check: AI Verification & Testing (CCQ 9)

An engineer prompts an AI to generate a complex payment calculation module, and then asks the same AI to write unit tests without providing a formal specification. All tests pass. What is the primary risk?

- **A.** Echo-chamber validation: The generated tests merely mirror the AI's internal flawed assumptions rather than actual business requirements.
- **B.** Performance bottleneck: AI-generated test assertions take significantly longer to execute than human-written assertions.
- **C.** Compilation failure: Testing frameworks cannot parse automated mock datasets generated by large language models.
- **D.** Version lock-in: The test suite becomes tightly coupled to a single specific cloud runtime environment.



Interactive Activity: The "Vibe Coding" Challenge

Classroom Poll & Discussion: AI in Practice

- **Poll:** When using AI coding assistants, how often do you inspect and understand every line before committing?
- **Discussion:** Suppose an AI assistant writes a 200-line asynchronous database handler that passes 2 basic tests. Is it safe to deploy? What verification steps must a professional engineer execute?



1.9 Frequently Asked Questions (FAQ) & Recap

> *"The most important tool in software engineering is the disciplined, critical human mind."* „

Frequently Asked Questions (FAQ)

- **Q1: Programming vs. Software Engineering?**
  - *Answer:* Programming is writing code. Software engineering is programming integrated over time, managing team collaboration, constraints (budget/schedule), quality attributes, and long-term evolution.
- **Q2: Why do correct programs with 100% test coverage fail?**
  - *Answer:* Software requires Data, Operational Procedures, and Documentation. Deficiencies in these non-code pillars or in Specification lead to failure.
- **Q3: Satisficing vs. Optimizing?**
  - *Answer:* Real-world constraints (time, budget) require a "satisficing" solution (sufficient to meet constraints) rather than a technically "optimized" one.
- **Q4: The risk of "Vibe Coding" with AI?**
  - *Answer:* Blindly accepting AI code without review leads to bugs. Prevent with code reviews, specification-first testing, and treating AI output as drafts.

Conceptual Recap: Fill-in-the-Blank Quiz

Word Bank: Documentation | Brooks's Law | Essential Difficulty | ISO/IEC 25010 | NATO Conference | Verification | Public Interest

1. According to the IEEE definition, software consists of programs, data, operational procedures, and [_____].
2. Adding manpower to a late software project makes it later is famously known as [_____].
3. In "No Silver Bullet," Fred Brooks argues that inherent software complexity is an [_____].
4. The [_____] standard defines 8 product quality characteristics including Security and Maintainability.
5. The term "Software Crisis" was formally recognized at the 1968 [_____] in Garmisch.
6. The question "Are we building the product right?" refers to [_____], while building the right product is Validation.
7. The first principle of the ACM/IEEE Software Engineering Code of Ethics prioritizes the [_____].

Interactive Mode: Click any blank slot to reveal or hide the answer!

References & Further Reading

- Sommerville, Ian. *Software Engineering* (10th Edition). Pearson. Official Website
- Brooks, Frederick P. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.
- ISO/IEC 25010:2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.
- ACM/IEEE Joint Task Force on Software Engineering Ethics. *Software Engineering Code of Ethics*. IEEE CS
- Brignull, Harry. *Deceptive Patterns: Exposing the Tricks Tech Companies Use to Control You*.